客户端逻辑安全审计报告

空洞骑士复刻 · 第0轮骨架(Web 单机版) — 存档完整性 / 进度伪造 / 客户端健壮性

目标	`https://a53d60dc08e38084e.app.workbuddy.host/?sharecode=5YAX54MVUC`
应用类型	纯客户端单机游戏(Canvas + 内联 JS,~1.16 MB 单文件,零外部资源)
托管平台	腾讯 CloudStudio Gateway(WorkBuddy 预览)
审计日期	2026-11-12
审计范围	游戏客户端逻辑:存档系统、进度 flag、护符装配、字段校验、客户端完整性
授权	资产方授权(自有/客户资产),范围限于上述客户端逻辑

1. 执行摘要

目标是一个**纯客户端、无后端、无网络调用**的单机游戏。经静态审计,未发现经典 Web 漏洞 (XSS / SQLi / SSRF / IDOR / 命令注入)的落点:应用无用户文本输入、无反射点、无危险 sink 接外部数据, 存档值仅作为索引查询硬编码数据表,不直接拼入 DOM。

核心风险集中在**客户端数据完整性**:存档以明文 JSON 存放于 `localStorage`, **无任何签名 / 校验和 / 防篡改**,玩家可任意伪造进度与资源。对纯单机而言当前无外部危害, 但这是唯一真实的完整性根因——一旦未来接入排行榜、速通验证或联机功能,将升级为高危作弊漏洞。 此外存在若干**健壮性缺口**(字段未钳制、flags 无白名单、护符装配未校验),可被恶意或损坏存档触发异常状态。

2. 风险总览

编号	发现	等级	一句话结论
F1	存档零完整性保护(客户端全信任)	中	明文存档无签名,可任意伪造进度/资源/结局
F2	进度 flags 合入无白名单、无类型校验	中	任意键值直接写入 `Game.flags`,字符串亦按真值判定
F3	护符装配加载时未校验所有权/槽位	中	可装备未拥有/超槽位护符,数值无条件生效
F4	部分字段未钳制(geo / soul / respawnRoom)	低	极端/非法值可导致崩溃或软锁
F5	存档版本号 `d.ver` 读取时被忽略	信息	无版本门控(设计为兼容读取)

评级说明:目标为**无在线功能**的单机游戏,不存在跨用户影响、无排行榜、无经济系统, 故最高定为「中」。若未来接入任何服务端/在线功能,F1、F2、F3 应整体上调至「高」。

3. 范围与测试方法

- **资产枚举:** 抓取目标首页与响应头,确认单文件静态应用、无外部资源、无网络调用。
- **攻击面扫描:** 检索 `fetch`/XHR/WebSocket、 `eval` / `innerHTML` / `document.write` 、 DOM XSS 源 (`location.*` / `document.cookie` / `referrer`)、用户输入控件。
- **存档系统审查:** 通读 `buildSaveData()` / `loadGame()` / `saveSignature()` / `recomputeCharms()` / `maxNotches()` 及 `Game.flags` 定义,逐字段核对钳制与校验逻辑。
- **数据流追溯:** 确认 `sharecode` 仅由平台网关处理(应用代码 0 处引用),不属于本应用逻辑范围。

4. 详细发现

F1 — 存档零完整性保护(客户端全信任)

中

位置: `saveGame()` (15968)、 `buildSaveData()` (15919)、 `saveSignature()` (15984)

存档 `localStorage["hk_save_v2"]` 为明文 JSON,写入前无 HMAC、无校验和、无加密。 `saveSignature()` 的名称具有误导性——它仅用于**变更检测**(避免同一帧重复写盘), 而非完整性校验。全项目检索未发现 `hmac / checksum / integrity / crypto.subtle / sha256` 等任何完整性机制。

影响: 玩家可直接编辑 `localStorage` 伪造吉欧、灵魂、血量上限、骨钉等级、全部进度 flag、 护符与结局标记。 `loadGame()` 信任存档各字段并合入运行态。

说明: 纯单机无外部危害;若未来接入排行榜/速通验证/联机,此为高危作弊漏洞。

F2 — 进度 flags 合入无白名单、无类型校验

中

位置: `loadGame()` 第 16027 行

```
Game.flags = Object.assign({}, FLAGS_DEFAULT, d.f || {});
```

`d.f` 中的任意键值被直接合并进 `Game.flags`,既无键名白名单,也无类型校验。 可伪造关键进度 flag:`bossDead2`(载入后自动授予 `hasDive` 下砸)、`superDash2`、`kingsBrand`、`hasVoidHeart`、`allGrubsSaved`、`ending_achieved` / `ending_type`、`colosseumCleared`、`dreamersDefeated`、`bought_*`(商店)、`notch_*`(槽位)。

次生问题:

- **值不布尔化:** 游戏全程按真值判断(`if (Game.flags.bossDead2)`),因此 `"false"`、`"0"` 等字符串亦为真值,导致逻辑误判。
- **原型污染面:** `d.f` 可携带 `__proto__` 键(经 `JSON.parse` 保留为自有属性), `Object.assign` 经 setter 将其设到 flags 对象原型上(局部污染,非全局,危害低但属健壮性缺口)。

F3 — 护符装配加载时未校验所有权 / 槽位

中

位置: `loadGame()` 第 16039 行、`recomputeCharms()`(9742)

```
Game.charmsEquipped = (d.ch || []).filter(id => CHARMS.some(c => c.id === id));
```

载入时仅过滤非法护符 id,**不校验是否拥有**(`oc`)、**不校验槽位成本**(`maxNotches()`)。 随后 `recomputeCharms()` 无条件按已装备列表应用全部数值:伤害叠乘至 ×2.25、无敌加成 +160、自动回血、空中二段冲等。

结果: 可装备未拥有的护符、可超出槽位上限(代价仅为 1 伤害/60 帧的过载反噬,见 13642 行)。

F4 — 部分字段未钳制

低

位置: `loadGame()` (16019)

字段	现状	风险
`geo` (g)、`soul` (s)	仅 `|0` 强转 int32,无上下界	**极大值回绕为负、负值可行**
`respawnRoom` (r)	未校验 `ROOMS[d.r]`	**可写入不存在的房间 id → 软锁/异常面**
`mhp` / `hp` / `nailLevel` / `bankGeo` / `shade`	均已钳制	**安全(对照项)**

说明:作者已对多数字段做了钳制,上述两处属遗漏。

F5 — 存档版本号 `d.ver` 读取时被忽略

信息

位置: `SAVE_VER = 3` (15887)、`loadGame()`

`SAVE_VER` 随存档写入,但读取时从不检查 `d.ver` (注释声明「版本不匹配不清空,走兼容读取」),无版本门控。

5. 复现验证(PoC)

在目标页面的 DevTools Console 执行以下代码,随后刷新页面(或直接改写 `localStorage` 后刷新):

```
const k = "hk_save_v2";
let s = JSON.parse(localStorage.getItem(k) || "{}");
s.g = 999999; s.nl = 99;                 // 吉欧、骨钉(被钳到 NAIL_MAX)
s.f = Object.assign(s.f || {}, {
  bossDead2: true, superDash2: true, canSuperDash: true,   // 下砸 / 超级冲刺
  kingsBrand: true, hasVoidHeart: true,                    // 深渊 / 结局能力
  allGrubsSaved: true, grubsMask: true,
  ending_achieved: true, ending_type: 5
});
s.ch = ["strength","fragile_power","soul_eater","dashmaster",
        "jonny","stalwart_shell","longbrand"];             // 全护符
localStorage.setItem(k, JSON.stringify(s));
```

预期结果:载入后拥有满级骨钉、下砸/超级冲刺/深渊/结局能力、全护符数值、全进度 flag 与高额吉欧, 证实存档可被任意伪造。该 PoC 基于 `loadGame()` / `buildSaveData()` 静态逻辑推导。

6. 修复建议(按优先级)

1. **补健壮性缺口(F4):** 为 `geo` / `soul` 增加范围钳制,`respawnRoom` 增加 `ROOMS` 校验。成本低,收益最直接。
2. **flags 白名单 + 布尔化(F2):** 在 `loadGame()` 对 `d.f` 做 `FLAGS_DEFAULT` 键名白名单过滤,布尔 flag 强制 `!!`;一并消除 `__proto__` 污染面与「字符串当真值」的误判。
3. **护符加载校验(F3):** 载入时按 `oc`(拥有)与 `maxNotches()` 过滤/卸载超限护符,对齐运行时 `charmOverload()` 语义。
4. **版本门控(F5,可选):** 引入 `d.ver` 检查与显式迁移分支。
5. **接入在线功能前的必要项(F1):** 客户端本地签名对可改的 JS 无效,需改用**服务端存档或服务端重算关键进度** + 版本门控;当前纯单机阶段可暂缓。

7. 边界说明

- 本报告仅覆盖应用自身客户端逻辑;托管平台 `workbuddy.host` / 腾讯 CloudStudio 的 `sharecode` 访问控制**不在范围内**。
- 未发现经典 Web 漏洞:无用户文本输入、无反射点、无网络调用、存档值仅作索引查硬编码表(不直接拼入 `innerHTML`)。
- 未审计游戏玩法逻辑本身(Boss 状态机、切房/传送、死亡惩罚等),如需要可另行开展。

报告由自动化安全审计助手基于静态代码分析生成,结论与 PoC 均依据 `loadGame()` / `buildSaveData()` 等源码推导。等级评定基于目标「纯单机、无在线功能」的实际风险面。